

❖ HANDS-ON 8

❖ HTTP Client — API Integration, Observables & Interceptors

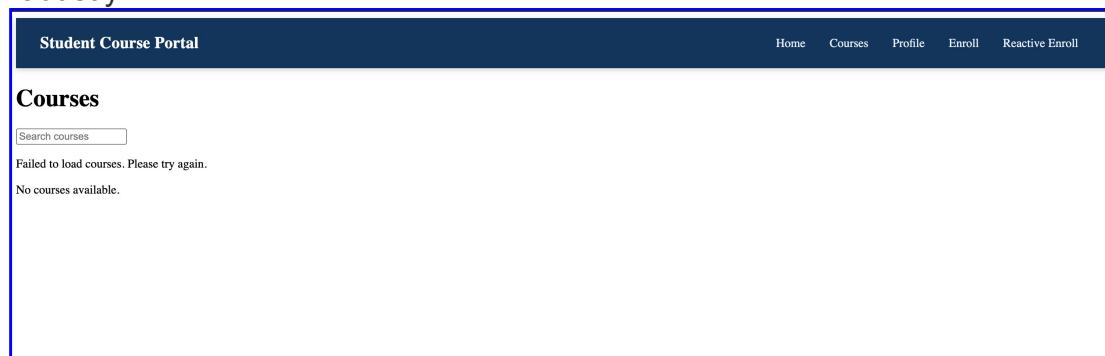
Task 1: Replace Service Data with HttpClient Calls

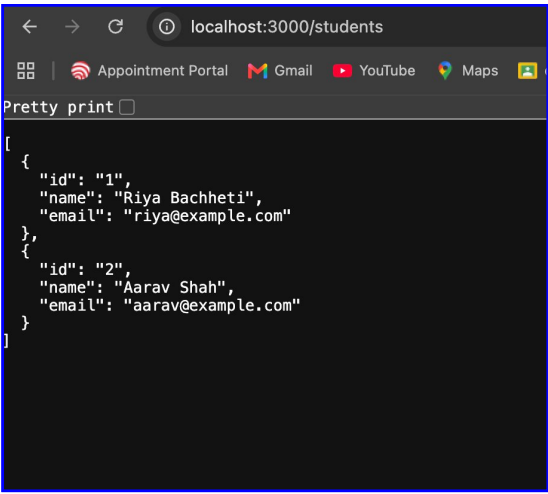
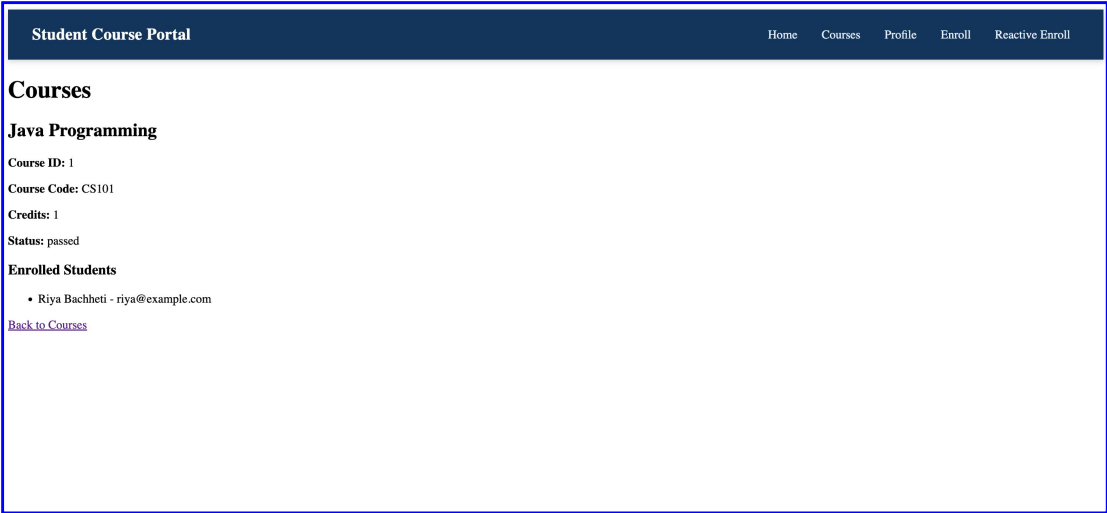
Goal: Refactor CourseService and EnrollmentService to use real HTTP calls.

```
TS course.ts U X TS auth-guard.ts U TS unsaved-changes-guard.ts U TS enrollment.routes.ts U
student-course-portal > src > app > services > TS course.ts > CourseService
1 import { Injectable } from '@angular/core';
2 import { HttpClient } from '@angular/common/http';
3 import { Observable } from 'rxjs';
4 import { Course } from '../models/course.model';
5
6 @Injectable({
7   providedIn: 'root'
8 })
9 export class CourseService {
10   private readonly apiUrl = 'http://localhost:3000/courses';
11
12   constructor(private http: HttpClient) {}
13
14   getCourses(): Observable<Course[]> {
15     return this.http.get<Course[]>(this.apiUrl);
16   }
17
18   getCourseById(id: number): Observable<Course> {
19     return this.http.get<Course>(`${this.apiUrl}/${id}`);
20   }
21
22   createCourse(course: Omit<Course, 'id'>): Observable<Course> {
23     return this.http.post<Course>(this.apiUrl, course);
24   }
25
26   updateCourse(course: Course): Observable<Course> {
27     return this.http.put<Course>({
28       `${this.apiUrl}/${course.id}`,
29       course
30     });
31   }
32
33   deleteCourse(id: number): Observable<void> {
34     return this.http.delete<void>(`${this.apiUrl}/${id}`);
35   }
36 }
```

Task 2: RxJS Operators and Error Handling

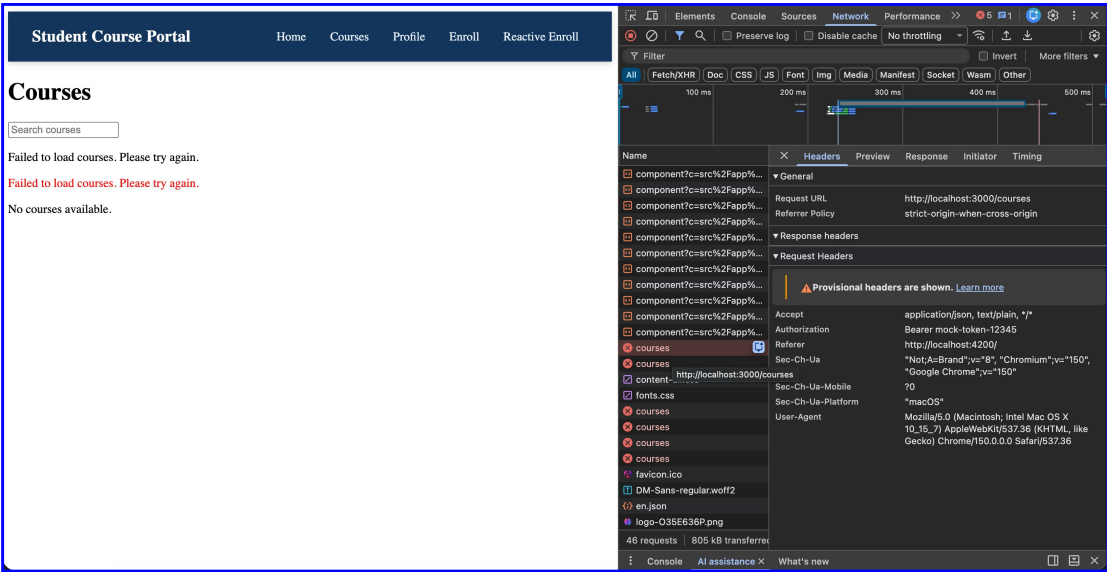
Goal: Apply RxJS operators to transform and handle HTTP responses robustly.





Task 3: HTTP Interceptors

Goal: Build interceptors to handle auth tokens and global error logging.



Java Programming

Status: passed

[Back to Courses](#)

The screenshot shows the Chrome DevTools Network tab. The 'FetchXHR' filter is applied. A list of requests is shown, with the first request 'component?c=src%2Fapp%' selected. The details pane on the right shows the 'General' tab for this request. The request is a GET to 'http://localhost:3000/courses/1' with a status of 200 OK. The response headers are visible, including 'Access-Control-Allow-Headers', 'Access-Control-Allow-Methods', 'Access-Control-Allow-Origin', 'Connection', 'Content-Length', 'Content-Type', 'Date', 'Keep-Alive', and 'X-Powered-By'. The request headers are also visible, including 'Accept', 'Accept-Encoding', and 'Accept-Language'.

Name	Headers	Preview	Response	Initiator	Timing
component?c=src%2Fapp%...	General				
component?c=src%2Fapp%...	Request URL	http://localhost:3000/courses/1			
component?c=src%2Fapp%...	Request Method	GET			
component?c=src%2Fapp%...	Status Code	200 OK			
component?c=src%2Fapp%...	Remote Address	[::1]:3000			
component?c=src%2Fapp%...	Referrer Policy	strict-origin-when-cross-origin			
component?c=src%2Fapp%...	Response headers	Raw			
component?c=src%2Fapp%...	Access-Control-Allow-Headers	content-type			
courses	Access-Control-Allow-Methods	GET, HEAD, PUT, PATCH, POST, DELETE			
content-all.css	Access-Control-Allow-Origin	*			
fonts.css	Connection	keep-alive			
favicon.ico	Content-Length	107			
DM-Sans-regular.woff2	Content-Type	application/json			
en.json	Date	Sat, 26 Jul 2026 19:27:45 GMT			
logo-530E636P.png	Keep-Alive	timeout=5			
1	X-Powered-By	tinyhttp			
enrollments?courseId=1	Request Headers	Raw			
enrollments?courseId=1	Accept	application/json, text/plain, *			
1	Accept-Encoding	gzip, deflate, br, zstd			
1	Accept-Language	en-US,en;q=0.9			
48 requests	808 kB transferred				